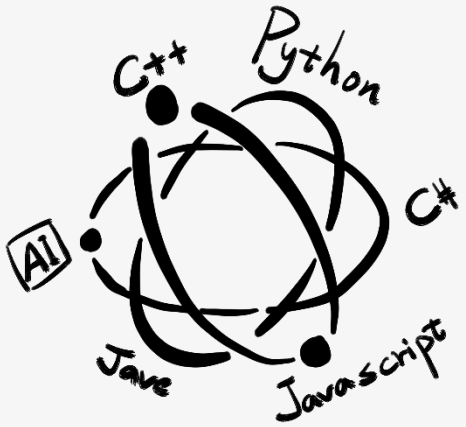
COOLMAKER
酷哥创客 人工智能教育





动态规划以及优化

酷哥创客 COOLMAKER







1 温故知新

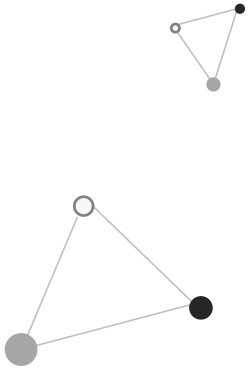
2 知识宝典

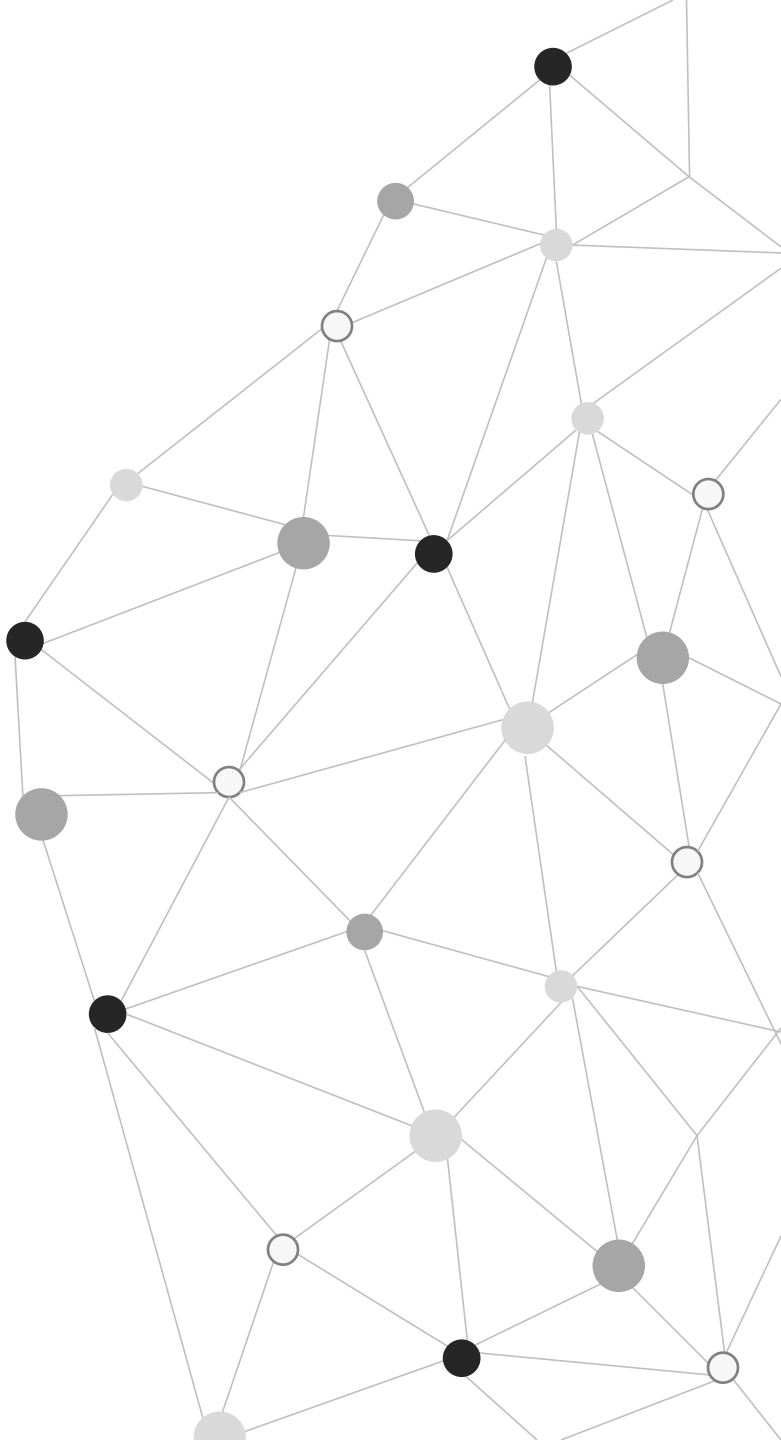
3 运用自如

4 回顾总结



知 识 宝 典







COOLMAKER
酷哥创客 人工智能教育

复杂动态规划

动态规划的状态与递推方程各不相同，大多是没有模板的，需要现场具体问题具体分析进行递推。

1. 最简单的就是最长上升/公共子序列、区间动态规划，背包问题的变种题（恰好、不少于、具体方案）等。
2. 复杂的有多维动态规划。
3. 数学、算法、数据结构和动态规划的结合，包括但不局限于等差、等比数列的求和公式等，如质数筛中递推，组合数，DAG上DP，树上DP
4. 动态规划的优化

所谓 DP 优化，是指许多**动态规划问题虽然容易列出朴素的状态转移方程，但直接计算往往效率低下，因此需要借助一些技巧来降低时间复杂度。**这些方法彼此联系紧密，常常包含相似的思想，且往往需要结合多种技巧共同使用。

最简单的就像背包问题利用滚动数组从二维优化为一维的空间优化、前缀和优化。

除此之外，还会考察动态规划的最值优化、二分/双指针优化、数据结构、数学（斜率优化，四边形不等式优化）等的时间优化。



多维动态规划



多维动态规划

COOLMAKER
酷哥创客 人工智能教育

动态规划本身通过将复杂问题分解为相互重叠的子问题来求解。
多维动态规划则进一步，使用多维数组（如二维、三维甚至更高维度）来定义状态，以描述更复杂的决策过程。



练 习 2

COOLMAKER 酷哥创客 人工智能教育

5412: 拼图游戏

题目描述

小酷宝最近迷上了拼图游戏。他有两块由不同字母组成的拼图块 A 和 B，现在他想把这两块拼图合并成一个更大的拼图 C，要求：

合并规则：拼图 C 由 A 和 B 合并，拼图 C 必须包含 A 和 B 的所有字母，C 的长度 $|C|$ 为 $|A|+|B|$，A 和 B 都作为**不重叠的子序列**出现 C 中（即 C 可以拆分成两个不相交的子序列，一个等于 A，另一个等于 B）。

对称性评分：拼图 C 的**对称性**定义为 C 中最长的**对称片段**（即回文子串）的长度。

小酷宝希望在所有可能的合并方式中，找到对称性最高的拼图 C，并计算其对称性评分。

输入

第一行一个整数 $T\ (1 \le T \le 50)$，表示测试数据的组数。
接下来 $2T$ 行，每两行分别给出字符串 A 和 B（仅包含小写字母，且 $1 \le |A|, |B| \le 50$）。

输出

对于每组数据，输出一行一个整数，表示所有可能的 C 的最大对称性评分。

样例输入1 复制

```
2
aa
bb
a
aaaabcaa
```

样例输出1 复制

```
4
5
```

COOLMAKER
酷哥创客 人工智能教育

定义四维 DP 数组：

$f[i][j][k][l]$ 表示用 A 的子串 $A[i..j]$ 和 B 的子串 $B[k..l]$ 能否组成一个回文串

状态转移考虑回文串两端的四种可能组合：

两端来自 A：要求 $A[i] = A[j]$ 且中间部分 $f[i+1][j-1][k][l]$ 为真

两端来自 B：要求 $B[k] = B[l]$ 且中间部分 $f[i][j][k+1][l-1]$ 为真

左端 A 右端 B：要求 $A[i] = B[l]$ 且中间部分 $f[i+1][j][k][l-1]$ 为真

左端 B 右端 A：要求 $B[k] = A[j]$ 且中间部分 $f[i][j-1][k+1][l]$ 为真

答案揭秘

COOLMAKER
酷哥创客 人工智能教育

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 60;
char a[N], b[N];
int T, f[N][N][N][N];
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin >> T;
    while(T--)
    {
        cin >> a + 1 >> b + 1;
        int len_a = strlen(a + 1), len_b = strlen(b + 1);
        int res = 1;
        for(int len1 = 0; len1 <= len_a; len1++)
            for(int len2 = 0; len2 <= len_b; len2++)
                for(int i = 1, j = len1; j <= len_a; i++, j++)
                    for(int k = 1, l = len2; l <= len_b; k++, l++)
                    {
                        if(len1 + len2 <= 1)
                            f[i][j][k][l] = 1;
                        else
                        {
                            f[i][j][k][l] = 0;
                            if(len1 > 1)
                                f[i][j][k][l] |= (f[i + 1][j - 1][k][l] && a[i] == a[j]);
                            if(len2 > 1)
                                f[i][j][k][l] |= (f[i][j][k + 1][l - 1] && b[k] == b[l]);
                            if(len1 && len2)
                            {
                                f[i][j][k][l] |= (f[i + 1][j][k][l - 1] && (a[i] == b[l]));
                                f[i][j][k][l] |= (f[i][j - 1][k + 1][l] && (a[j] == b[k]));
                            }
                        }
                        if(f[i][j][k][l])
                            res = max(res, len1 + len2);
                    }
        cout << res << "\n";
    }
    return 0;
}
```

2487: 括号序列

题目描述

小 w 在赛场上遇到了这样一个题：一个长度为 n 且符合规范的括号序列，其有些位置已经确定了，有些位置尚未确定，求这样的括号序列一共有多少个。

身经百战的小 w 当然一眼就秒了这题，不仅如此，他还觉得一场正式比赛出这么简单的模板题也太小儿科了，于是他把这题进行了加强之后顺手扔给了小 c。

具体而言，小 w 定义“超级括号序列”是由字符 `(` 、 `)` 、 `*` 组成的字符串，并且对于某个给定的常数 k ，给出了“符合规范的超级括号序列”的定义如下：

1. `()` 、 `(S)` 均是符合规范的超级括号序列，其中 `S` 表示任意一个仅由**不超过 个**字符 `*` 组成的非空字符串（以下两条规则中的 `S` 均为此含义）；

2.如果字符串 `A` 和 `B` 均为符合规范的超级括号序列，那么字符串 `AB` 、 `ASB` 均为符合规范的超级括号序列，其中 `AB` 表示把字符串 `A` 和字符串 `B` 拼接在一起形成的字符串；

3.如果字符串 `A` 为符合规范的超级括号序列，那么字符串 `(A)` 、 `(SA)` 、 `(AS)` 均为符合规范的超级括号序列。

4.所有符合规范的超级括号序列均可通过上述 3 条规则得到。

例如，若 k = 3 ，则字符串 `((**()*(*))*)(***)` 是符合规范的超级括号序列，但字符串 `*()` 、 `(*()*)` 、 `((**))*)` 、 `(****(*))` 均不是。特别地，空字符串也不被视为符合规范的超级括号序列。

现在给出一个长度为 n 的超级括号序列，其中有一些位置的字符已经确定，另外一些位置的字符尚未确定（用 `?` 表示）。小 w 希望能计算出：有多少种将所有尚未确定的字符一一确定的方法，使得得到的字符串是一个符合规范的超级括号序列?

可怜的小 c 并不会做这道题，于是只好请求你来帮忙。

输入

第一行，两个正整数 n, k 。

对于 100% 的数据，$1 \leq k \leq n \leq 500$。

第二行，一个长度为 n 且仅由 `(` 、 `)` 、 `*` 、 `?` 构成的字符串 S 。

输出

输出一个非负整数表示答案对 $10^9 + 7$ 取模的结果。

样例输入1

```
7 3
(*??*??
```

样例输出1

```
5
```

【样例解释 #1】

如下几种方案是符合规范的：

1. `(**)*()`
2. `(**(*))`
3. `(*(**))`
4. `(*)**()`
5. `(*)(**)`

【数据范围】

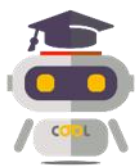
测试点编号	$n \leq$	特殊性质
$1 \sim 3$	15	无
$4 \sim 8$	40	无
$9 \sim 13$	100	无
$14 \sim 15$	500	S 串中仅含有字符 `?`
$16 \sim 20$	500	无

对于 100% 的数据，$1 \leq k \leq n \leq 500$。

对于测试点 $1 \sim 3$ （15分）

枚举所有 `?` 的三种放置可能，然后检验是否合法。

最多有 3^{15} 种可能，检验括号序列是否合法需要 $O(n)$ 的时间，总时间复杂度 $O(n \cdot 3^{15})$，预计得分 15 分。



时间复杂度 $O(3^n \times n)$，期望得分 15pts。

```cpp
#include<bits/stdc++.h>
using namespace std;
int n,k,ans;
char s[505];
bool check()
{...}
void dfs(int x)
{
    if(x==n+1)
    {
        if(check())ans++;
        return ;
    }
    if(s[x]!='?')dfs(x+1);
    else
    {
        s[x]='(';
        dfs(x+1);
        s[x]=')';
        dfs(x+1);
        s[x]='*';
        dfs(x+1);
        s[x]='?';
    }
}
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin>>n>>k>>s+1;
    dfs(1);
    cout<<ans<<"\n";
    return 0;
}
```

```cpp
bool check()
{
    vector<int>pre(n+2),suf(n+2);;
    vector<char>plast(n+1),slast(n+1);;
    for(int i=1;i<=n;i++)
    {
        if(s[i]=='*')pre[i]=pre[i-1]+1;
        else pre[i]=0;
        if(pre[i]>k)return false;
        if(i<n)
        {
            if(s[i]!='*')plast[i+1]=s[i];
            else plast[i+1]=plast[i];
        }
    }
    for(int i=n;i>=1;i--)
    {
        if(s[i]=='*')suf[i]=suf[i+1]+1;
        else suf[i]=0;
        if(i>1)
        {
            if(s[i]!='*')slast[i-1]=s[i];
            else slast[i-1]=slast[i];
        }
    }
    stack<int>st;
    for(int i=1;i<=n;i++)
    {
        if(s[i]=='(')
        {
            st.push(i);
        }
        else if(s[i]==')')
        {
            if(st.empty())
            {
                return false;
            }else
            {
                if(st.top()+4<i && suf[st.top()+1] !=i-st.top()-1 && suf[st.top()+1] && pre[i-1])
                {
                    return false;
                }
                st.pop();
            }
        }
    }
    if(pre[1] || suf[n] || !st.empty())
        return false;
    return true;
}
```

S：不超过 k 个 * 组成
A：合法序列

根据题目描述，符合要求的括号序列可以分成两类

1. 包含型：$(), (A), (S), (AS), (SA)$
2. 并列型：AA, ASA

于是我们就可以开始逐一枚举这几种情况并统计答案了！



S:不超过 k 个*组成
A：合法序列
包含关系 () (S) (A) (SA) (AS)
并列关系 AA ASA

DP[0]: S

```
if(len<=k && dp[l][r-1][0] && (s[r]=='*' || s[r]=='?'))dp[l][r][0]=1;
```

DP[1]: AS
DP[2]: SA
DP[3]：包含关系 () (S) (A) (SA) (AS)
DP[4]：并列关系 AA ASA　　//最左右括号不能匹配上
DP[5]：A：DP[3]+DP[4]



S:不超过 k 个*组成
A：合法序列
包含关系 () (S) (A) (SA) (AS)
并列关系 AA ASA

```
for(int k=l;k<r;k++)
{
    dp[l][r][1]+=dp[l][k][5]*dp[k+1][r][0];
    dp[l][r][2]+=dp[l][k][0]*dp[k+1][r][5];
    dp[l][r][1]%=mod;
    dp[l][r][2]%=mod;
}
```

DP[0]: S
DP[1]: AS
DP[2]: SA
DP[3]：包含关系 () (S) (A) (SA) (AS)
DP[4]：并列关系 AA ASA　　//最左右括号不能匹配上
DP[5]：A：DP[3]+DP[4]



S:不超过 k 个*组成
A：合法序列
包含关系 () (S) (A) (SA) (AS)
并列关系 AA ASA

```
if((s[l]=='(' || s[l]=='?') && (s[r]==')' || s[r]=='?'))
{
    dp[l][r][3]=dp[l+1][r-1][0]+dp[l+1][r-1][5]+dp[l+1][r-1][1]+dp[l+1][r-1][2];
    dp[l][r][3]%=mod;
}
```

DP[0]: S
DP[1]: AS
DP[2]: SA
DP[3]：包含关系 () (S) (A) (SA) (AS)
DP[4]：并列关系 AA ASA　　//最左右括号不能匹配上
DP[5]：A：DP[3]+DP[4]



S:不超过 k 个*组成
A：合法序列
包含关系 () (S) (A) (SA) (AS)
并列关系 AA ASA

```
for(int k=l+1;k<r-1;k++)
{
    dp[l][r][4]+=dp[l][k][3]*(dp[k+1][r][2]+dp[k+1][r][5]);
    dp[l][r][4]%=mod;
}
```

DP[0]: S
DP[1]: AS
DP[2]: SA
DP[3]：包含关系 () (S) (A) (SA) (AS)
DP[4]：并列关系 AA ASA　　//最左右括号不能匹配上，需要注意去重
DP[5]：A：DP[3]+DP[4]



```cpp
#include<bits/stdc++.h>
using namespace std;
const int mod=1e9+7;
int n, k;
char s[505];
long long dp[505][505][6];
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin>>n>>k>>s+1;
    for(int i=1; i<=n; i++)
    {
        if(s[i]=='*' || s[i]=='?') dp[i][i][0]=1;
        if(k>1 && i<n && (s[i]=='*' || s[i]=='?') && (s[i+1]=='*' || s[i+1]=='?')) dp[i][i+1][0]=1;
        if(i<n && (s[i]=='(' || s[i]=='?') && (s[i+1]==')' || s[i+1]=='?'))dp[i][i+1][5]=dp[i][i+1][3]=1;
    }
    for(int len=3; len<=n; len++)
    {
        for(int l=1, r=len; r<=n; l++, r++)
        {
            if(len<=k && dp[l][r-1][0] && (s[r]=='*' || s[r]=='?'))dp[l][r][0]=1;
            for(int k=l; k<r; k++)
            {
                dp[l][r][1]+=dp[l][k][5]*dp[k+1][r][0];
                dp[l][r][2]+=dp[l][k][0]*dp[k+1][r][5];
                dp[l][r][1]%=mod;
                dp[l][r][2]%=mod;
            }
            if((s[l]=='(' || s[l]=='?') && (s[r]==')' || s[r]=='?'))
            {
                dp[l][r][3]=dp[l+1][r-1][0]+dp[l+1][r-1][5]+dp[l+1][r-1][1]+dp[l+1][r-1][2];
                dp[l][r][3]%=mod;
            }
            for(int k=l+1; k<r-1; k++)
            {
                dp[l][r][4]+=dp[l][k][3]*(dp[k+1][r][2]+dp[k+1][r][5]);
                dp[l][r][4]%=mod;
            }
            dp[l][r][5]=dp[l][r][3]+dp[l][r][4];
            dp[l][r][5]%=mod;
        }
    }
    cout<<dp[1][n][5]<<"\n";
    return 0;
}
```

COOLMAKER
酷哥创客 人工智能教育



bitset优化





COOLMAKER 酷哥创客 人工智能教育

【bitset】

bitset容器主要是用来压缩表示布尔数组，常用于位运算。我们在后面学习一些难的算法的时候，经常会用到位运算的知识点，来帮我们减少时间复杂度，n位bitset执行一次位运算时间复杂度为n/32或者n/64。

bitset这个容器的强大之处在于，它既可以像数组的方式进行随机访问，修改。也可以看成一个数，像变量一样进行位运算，进行进制转换。

1.bitset的定义

```
//定义一个大小为4的bitset,初始值所有位都为0
bitset<4> op;
```

2.以数字进行初始化

```
//将数字4转化为二进制100
//不足四位前面补零 op1=0100
bitset<4> op1(4);
//将数字17转化为二进制10001
//超过四位,只取后面四位 op2=0001
bitset<4> op2(17);
```

3.以字符串进行初始化

```
//长度小前面用0补齐 op3=0001
bitset<4> op3("001");
//长度大只取前面4位 op4=1100
bitset<4> op4("110001");
```

4.bitset的赋值、修改、运算操作

```
//可以通过数字直接赋值的方式修改bitset
op1 = 15;
//数组下标是从后往前，也可以理解成最低位为0
op2[1] = 1;
cout<<op2<<endl;
//可以直接进行位运算
cout<<(op1 >> 1)<<endl;
cout<<(op2 << 2)<<endl;
cout<<(op1|op2)<<endl;

cin>>op1; //也可以直接将01串输入
```

```
C:\Users\Administrator\De
0011
0111
1100
1111
```

C++ 空间复杂度

COOLMAKER
酷哥创客 人工智能教育

bitset<N> 的大小 N 是一个模板参数，**必须是一个编译时常量**，其内部通常使用一个或多个整型数组（如 unsigned long 或 unsigned long long）来存储位数据。
虽然实际占用会因编译器和平台而异，但可以用以下公式估算**理论最小内存占用**（以字节为单位）：
理论最小字节数 ≈ ceil(N / 8.0)

为了访问效率和硬件优化，编译器会对 bitset 进行内存对齐因此，其实际内存分配通常以机器字长（如 4 字节或 8 字节），会是机器字长的倍数，因此往往大于或等于这个理论最小值

C++ 时间复杂度

批量操作与位运算，这是 bitset 的核心优势所在bitset 的位运算（如按位与 &、按位或 |、按位异或 ^、按位取反 ~、移位 <<, >>）是通过底层硬件指令实现的，一次操作可以处理 32 位、64 位甚至更多（取决于硬件和实现）。

这意味着**完成一次对整个容器的位运算**，bitset 的**时间复杂度大致是 O(N / k)**，其中 k 是机器字长（如 32 或 64），而不是 bool 数组需要的 O(N)。这在处理大规模数据时能带来数量级级别的性能提升，由于一次处理多个位，**速度提升往往可达数十倍。**

bitset由于可以节省时间复杂度这一特点，经常被应用于各种算法来优化代码，如动态规划问题等

COOLMAKER 酷哥创客 人工智能教育

bitset的常用函数。

函数名称	函数作用	时间复杂度
set()/set(i)	将所有位设置成1/将第i位设置成1	O(N/64)/O(1)
reset()/reset(i)	将所有位设置成0/将第i位设置成0	O(N/64)/O(1)
count()	返回所有位中1的个数	O(N/64)
filp()/flip(i)	翻转所有的位//将第i位翻转	O(N/64)/O(1)
to_ullong	将二进制转为十进制数 返回十进制数（unsigned long long）	O(N/64)
any()/none()	所有位都为0，any()返回false，none()返回true 至少有一位为1，any()返回true，none()返回false	O(N/64)
size()	返回bitset的长度	O(1)

5010: 连接召唤



题目描述

小酷宝在玩游戏王，他正在学习连接召唤的相关规则。

连接召唤的本质可以简单地理解成，根据连接怪兽的link值（卡面右下角的数字），提供总量为对应数量的怪兽作为素材进行特殊召唤的一种方式。特别的，如果有连接怪兽被作为素材，它可以被视作一个普通素材，也可以被视为数量和自己的link值相同的素材怪兽。

举个例子，要召唤一只 LINK-4 的怪兽，素材可以是场上的任意 4 只怪兽（$1+1+1+1=4$），或者 2 只 LINK-2 怪兽（$2+2=4$），或者 1 只 LINK-3 怪兽以及 1 只 LINK-2 怪兽（$3+1=4$，其中 LINK-2 怪兽被作为普通素材）。

现在小酷宝想知道，使用当前场上的怪兽作为素材，有哪些连接怪兽可以被召唤？（每次询问不会真的消耗素材去召唤）

输入

本题包含多组测试数据。

输入的第一行包含一个正整数 T，表示测试数据的组数。

每组数据第一行两个正整数 n, m，分别表示当前场上的怪兽和准备被召唤的怪兽。

第二行 n 个正整数 $a_1, a_2, \cdots, a_n$，表示场上怪兽的link值。

第三行 m 个正整数 $b_1, b_2, \cdots, b_m$，表示被召唤怪兽的link值。

对于 50% 的数据，保证 $1 \le T \le 5$，$1 \le n, m \le 10$，$1 \le a_i, b_i \le 2000$。

对于 100% 的数据，保证 $1 \le T \le 5$，$1 \le n, m \le 10^3$，$1 \le a_i, b_i \le 10^5$。

输出

每组数据输出 m 行，对于每组数据的第 i 行，如果第 i 只怪兽可以被召唤，输出 `YES`，否则输出 `NO`。

样例输入1 复制

```
1
6 3
8 3 12 7 9 7
24 50 13
```

样例输出1 复制

```
YES
NO
YES
```

一道典型的01背包题。

设DP状态 $f_{i,j}$ 为在只能使用前 i 只怪兽的情况下，是否能召唤link值为 j 的怪兽。

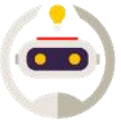
第 i 只怪兽可以①选择不使用、②当成 1 普通素材、③当成 a_i 素材，对应的状态分别是 $f_{i-1,j}$、$f_{i-1,j-1}$、$f_{i-1,j-a_i}$

故：$f_{i,j} = f_{i-1,j} \mid f_{i-1,j-1} \mid f_{i-1,j-a_i}$

时间复杂度 $O(nB+m)$，B = 最大link值。

```cpp
#include <bits/stdc++.h>
using namespace std;
int n, m, x;
bool f[100005];
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    int T;
    cin >> T;
    while(T--)
    {
        memset(f, 0, sizeof(f));
        f[0] = 1;
        cin >> n >> m;
        for(int i = 1; i <= n; i++)
        {
            cin>>x;
            for(int j = 100000; j >= 1; j--)
            {
                f[j] |= f[j - 1];
                if(j >= x)
                    f[j] |= f[j - x];
            }
        }
        for(int i = 1; i <= m; i++)
        {
            cin>>x;
            cout<<(f[x] ? "YES" : "NO")<<"\n";
        }
    }
    return 0;
}
```





COOLMAKER
酷哥创客 人工智能教育

bitset优化

注意到动态规划转移方程是一个 按位或运算，

可以使用bitset优化，将时间复杂度降低为 $O(\frac{nB}{64}+m)$

```cpp
#include <bits/stdc++.h>
using namespace std;
int n, m, x;
bool f[100005];
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    int T;
    cin >> T;
    while(T--)
    {
        memset(f, 0, sizeof(f));
        f[0] = 1;
        cin >> n >> m;
        for(int i = 1; i <= n; i++)
        {
            cin>>x;
            for(int j = 100000; j >= 1; j--)
            {
                f[j] |= f[j - 1];
                if(j >= x)
                    f[j] |= f[j - x];
            }
        }
        for(int i = 1; i <= m; i++)
        {
            cin>>x;
            cout<<(f[x] ? "YES" : "NO")<<"\n";
        }
    }
    return 0;
}
```

结果	内存	耗时
正确	1764	117
正确	1760	1677

```cpp
#include <bits/stdc++.h>
using namespace std;
int n, m, x;
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    int T;
    cin >> T;
    while(T--)
    {
        bitset<100001>f;
        f[0] = 1;
        cin >> n >> m;
        for(int i = 1; i <= n; i++)
        {
            bitset<100001>f2;
            cin>>x;
            f2|=f;
            f2|=f<<1;
            f2|=f<<x;
            swap(f,f2);
        }
        for(int i = 1; i <= m; i++)
        {
            cin>>x;
            cout<<(f[x] ? "YES" : "NO")<<"\n";
        }
    }
    return 0;
}
```



二分、双指针优化





4943: 地铁套票

加入错题本

时间限制: 1.0 Sec 内存限制: 128 MB

提交: 0 解决: 0

[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestData]

难度

★★★★☆

题目描述

小酷宝最近要参加一系列的编程比赛，为此他需要搭乘 n 次地铁.

地方地铁APP推出了 m 种套票，第 i 种套票需要 x_i 元，激活后可以在 y_i 时间内任意搭乘地铁（如果在时刻 t 激活，则可以在时刻 $[t \quad , t+y_i-1]$ 搭乘地铁，搭乘地铁的耗费的时间忽略不计，如果你在 $t+y_i-1$ 时刻搭乘地铁，那么你依然可以到达终点出站）。

小酷宝很抠门，他想知道如果只考虑前 i 次搭乘，最少只需多少金额。

输入

第一行有两个正整数 n, m ——搭乘地铁的次数和套票的种类。

第二行 n 个整数 t_i，表示小酷宝的第 i 次搭乘地铁的时刻。保证时刻各不相同，且从小到大递增。

接下来 m 行，每行两个正整数 x_i, y_i，表示第 i 个套票的价格和激活时间。

对于 30% 的数据，保证 $1 \le n \times m \le 10^3, 0 \le t_i \le 10^4$。

对于 50% 的数据，保证 $1 \le n \times m \le 10^5, 1 \le n^2 \times m \le 10^7$。

对于 100% 的数据，保证 $1 \le n \times m \le 10^6, 0 \le t_i \le 10^9, 1 \le x_i, y_i \le 10^9$。

样例输入1 复制

```
3 3
3 29 31
10 16
19 20
13 14
```

样例输出1 复制

```
10 20 20
```

输出

输出 n 个数，第 i 个数表示只考虑前 i 次搭乘最少需要的金额。

设 dp_i 为前 i 次搭乘最少需要的金额。考虑如果本次搭乘处于第 j 种套票的激活时间，则该套票在 $t_i - y_i + 1$ 时刻激活最划算，因此状态转移方程为 $dp_i = dp_{pos} + y_j$。其中 pos 是最近的满足 $t_{pos} < t_i - y_i + 1$ 的位置，可以通过二分或双指针求出。

时间复杂度 $O(nm)$，$O(nm \log n)$ 通过也可以。

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef unsigned long long ull;
int n,m;
int t[2000005],x[2000005],y[2000005];
ll dp[2000005];
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin>>n>>m;
    for(int i=1;i<=n;i++)
    {
        dp[i]=1e18;
        cin>>t[i];
    }
    for(int i=1;i<=m;i++)
    {
        cin>>x[i]>>y[i];
    }
    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=m;j++)
        {
            int k=1;
            while(t[k]<t[i]-y[j]+1)//暴力找
            {
                k++;
            }
            dp[i]=min(dp[i],dp[k-1]+x[j]);
        }
        cout<<dp[i]<<" ";
    }
    return 0;
}
```

```cpp
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
int n, m;
int t[2000005],x[2000005],y[2000005];
ll dp[2000005];
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n>>m;
    for(int i=1;i<=n;i++)
    {
        dp[i]=1e18;
        cin>>t[i];
    }
    for(int i=1;i<=m;i++)
    {
        cin>>x[i]>>y[i];
    }
    vector<int> lst(m, 1);
    for(int i=1; i<=n; i++)
    {
        for(int j=0; j<m; j++)
        {
            while(t[lst[j]]+y[j]<=t[i])
            {
                lst[j]++;
            }
            dp[i] = min(dp[i], dp[lst[j]-1]+x[j]);
        }
        cout<<dp[i]<<' ';
    }
    return 0;
}
```



前缀和优化



4643: 接龙（chain）

题目描述

在玩惯了成语接龙之后，小 J 和他的朋友们发明了一个新的接龙规则。

总共有 n 个人参与这个接龙游戏，第 i 个人会获得一个整数序列 S_i 作为他的词库。一次游戏分为若干轮，每一轮规则如下：

n 个人中的某个人 p 带着他的词库 S_p 进行接龙。若这不是游戏的第一轮，那么这一轮进行接龙的人不能与上一轮相同，但可以与上上轮或更往前的轮相同。

接龙的人选择一个长度在 $[2, k]$ 的 S_p 的连续子序列 A 作为这一轮的**接龙序列**，其中 k 是给定的常数。若这是游戏的第一轮，那么 A 需要以元素 1 开头，否则 A 需要以上一轮的接龙序列的最后一个元素开头。

序列 A 是序列 S 的连续子序列当且仅当可以通过删除 S 的开头和结尾的若干元素（可以不删除）得到 A 。

为了强调合作，小 J 给了 n 个参与游戏的人 q 个任务，第 j 个任务需要这 n 个人进行一次游戏，在这次游戏里进行恰好 r_j 轮接龙，且最后一轮的接龙序列的最后一个元素恰好为 c_j 。为了保证任务的可行性，小 J 请来你判断这 q 个任务是否可以完成的，即是否存在一个可能的游戏过程满足任务条件。

输入

本题有多组测试数据。

输入的第一行包含一个正整数 T ，表示数据组数。

接下来包含 T 组数据，每组数据的格式如下：

第一行包含三个整数 n, k, q ，分别表示参与游戏的人数、接龙序列长度上限以及任务个数。

接下来 n 行：

第 i 行包含 $(l_i + 1)$ 个整数 $l_i, S_{i,1}, S_{i,2}, \cdots, S_{i,l_i}$ ，其中第一个整数 l_i 表示序列 S_i 的长度，接下来 l_i 个整数描述序列 S_i 。

接下来 q 行：

第 j 行包含两个整数 r_j, c_j ，描述一个任务。

输出

对于每个任务：输出一行包含一个整数，若任务可以完成输出 `1` ，否则输出 `0` 。

样例输入1 复制

```
1
3 3 7
5 1 2 3 4 1
3 1 2 5
3 5 1 6
1 2
1 4
2 4
3 4
6 6
1 1
7 7
```

样例输出1 复制

```
1
0
1
0
1
0
0
```

[数据范围]

对于所有测试数据，保证：

- $1 \leq T \leq 5$；
- $1 \leq n \leq 10^5, 2 \leq k \leq 2 \times 10^5, 1 \leq q \leq 10^5$；
- $1 \leq l_i \leq 2 \times 10^5, 1 \leq S_{i,j} \leq 2 \times 10^5$；
- $1 \leq r_j \leq 10^2, 1 \leq c_j \leq 2 \times 10^5$；
- 设 $\sum l$ 为**单组测试数据**内所有 l_i 的和，则 $\sum l \leq 2 \times 10^5$。

测试点	$n \leq$	$r \leq$	$\sum l \leq$	$q \leq$	特殊性质
1	10^3	1	$2{,}000$	10^3	无
$2, 3$	10	5	20	10^2	
$4, 5$	10^3	10	$2{,}000$	10^3	A
6	10^5	10^2	2×10^5	10^5	
$7, 8$	10^3	10	$2{,}000$	10^3	B
$9, 10$	10^5	10^2	2×10^5	10^5	
$11, 12$	10^3	10	$2{,}000$	10^3	C
$13, 14$	10^5	10^2	2×10^5	10^5	
$15 \sim 17$	10^3	10	$2{,}000$	10^3	无
$18 \sim 20$	10^5	10^2	2×10^5	10^5	

特殊性质A：保证 $k = 2 \times 10^5$。

特殊性质B：保证 $k \leq 5$。

特殊性质C：保证在单组测试数据中，任意一个字符在词库中出现次数之和均不超过 5。

对于测试点 $2 \sim 3$ （10分）

定义`dis[i][j][k]`代表第 i 轮第 j 个人以数字 k 为结尾是否可能，bfs/dp 遍历所有接龙的可能性。

时间复杂度 $O(q \times r \times \sum l \times \sum l)$

定义dp[i][j]：代表第 i 轮能否以 j 字母结尾

缺少一维信息，怎么解决上一轮以 j 字母结尾接龙的人不是这轮接龙的人？

核心：

当上一轮存在一个以牌面值为 j 作为结尾的人，那么我们在考虑 j 是否可以作为开头时，需要考虑 j 的拥有者和上一轮 j 结尾的人是不是同一个人；如果上一轮 j 结尾的人数超过 1人那么一定可以。

```
for( 第几轮接龙 )
    for( 枚举这轮要接龙的人 A )
        for( 枚举本轮 A 要接龙的结尾字母 )
            for( 枚举 A 的结尾字母之前 k-1 个字母中是否存在上轮的结尾字母 )
```

时间复杂度 $O(r \times \sum (l_a \times l_a))$，期望得分 70pts。

70 分

```
void solve()
{
    cin >> n >> k >> q;
    vector<int>l(n + 1);
    vector<vector<int>>s(n + 1);
    vector<vector<int>>dp(101, vector<int>(200001, -1)); //第 i 轮能否以字母 j 结尾
    for(int i = 1; i <= n; i++)
    {
        cin >> l[i];
        s[i] = vector<int>(l[i] + 1);
        for(int j = 1; j <= l[i]; j++)
            cin >> s[i][j];
    }
    dp[0][1] = 0;
    for(int i = 1; i <= 100; i++)
    {
        for(int j = 1; j <= n; j++)
        {
            for(int t = 1; t <= l[j]; t++)
            {
                int S = s[j][t];
                for(int t2 = t - 1; t2 >= 1 && t2 >= t - k + 1; t2--)
                {
                    int S2 = s[j][t2];
                    if(dp[i - 1][S2] == 0 || (dp[i - 1][S2] != -1 && dp[i - 1][S2] != j)) //上一轮能否以S2结尾
                    {
                        if(dp[i][S] == -1)
                            dp[i][S] = j;
                        else if(dp[i][S] != j)
                            dp[i][S] = 0;
                        break;
                    }
                }
            }
        }
    }
    while(q--)
    {
        cin >> r >> c;
        cout << (dp[r][c] != -1) << "\n";
    }
}
```





COOLMAKER
酷哥创客 人工智能教育

for(第几轮接龙)
    for(枚举这轮要接龙的人 A)
        for(枚举本轮 A 要接龙的结尾字母)
            for(枚举 A 的结尾字母之前 k-1 个字母中是否存在上轮的结尾字母)

时间复杂度 $O(100 \times \sum (l_a \times l_a))$

额外定义sum[i][j]：代表能否以第 i个人的第 j 个字母开头
前缀和做差计算前 k-1 个字母中是否存在上轮 B 的结尾字母

for(第几轮接龙)
    for(枚举这轮要接龙的人 A)
        for(枚举本轮 A 要接龙的结尾字母)

时间复杂度 $O(100 \times \sum l_a)$

100 分

```cpp
void solve()
{
    cin >> n >> k >> q;
    vector<int>l(n + 1);
    vector<vector<int>>s(n + 1);
    vector<vector<int>>dp(101, vector<int>(200001, -1)); //第 i 轮能否以字母 j 结尾
    for(int i = 1; i <= n; i++)
    {
        cin >> l[i];
        s[i] = vector<int>(l[i] + 1);
        for(int j = 1; j <= l[i]; j++)
            cin >> s[i][j];
    }
    dp[0][1] = 0;
    for(int i = 1; i <= 100; i++)
    {
        for(int j = 1; j <= n; j++)
        {
            vector<int>sum(l[j] + 1);//第 i 轮能否以 j 的第 t 个字母开头
            for(int t = 1; t <= l[j]; t++)
            {
                int S = s[j][t];
                if(dp[i - 1][S] == 0 || (dp[i - 1][S] != -1 && dp[i - 1][S] != j)) // 能否以S开头
                    sum[t] = sum[t - 1] + 1;
                else
                    sum[t] = sum[t - 1];
                if(sum[t - 1] - sum[max(0, t - k)] > 0) // 前k-1个字母是否存在能开头的
                {
                    if(dp[i][S] == -1)
                        dp[i][S] = j;
                    else if(dp[i][S] != j)
                        dp[i][S] = 0;
                }
            }
        }
    }
    while(q--)
    {
        cin >> r >> c;
        cout << (dp[r][c] != -1) << "\n";
    }
}
```

```cpp
#include <bits/stdc++.h>
using namespace std;
int T, n, k, q, r, c;
void solve()
{...}
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> T;
    while(T--)
        solve();
    return 0;
}
```



极差优化





编辑距离3

COOLMAKER
酷哥创客 人工智能教育

题目描述

小酷宝的魔法糖果工厂最近研发了一种新型变形糖果，每颗糖果都能通过魔法变形操作改变自身的颜色或形状。现在工厂收到两份糖果序列样本：初始序列 S 和目标序列 T （每个序列由不同颜色的糖果组成）。

小酷宝需要在最多 K 次魔法操作内，将 S 变成与 T 完全相同的序列。每次操作可选择以下三种魔法之一：

插入魔法：在 S 序列任意位置添加一颗新糖果（颜色任选）；

删除魔法：移除 S 序列中的一颗糖果；

变色魔法：将 S 序列中的一颗糖果的颜色改为其他颜色。

输入

第一行：整数 K

第二行：字符串 S ，表示初始糖果序列（每个字符代表一颗糖果的颜色）；

第三行：字符串 T ，表示目标糖果序列。

数据范围：

$1 \leq K \leq 20$

序列 S 和 T 长度不超过 5×10^5 ，且仅包含小写字母（代表 26 种颜色）。

输出

若能在 K 次操作内将 S 变为 T ，输出 `Yes` ，否则输出 `No` 。

编辑距离，也称 Levenshtein 距离，指两个字符串之间互相转换的最少修改次数，通常用于在信息检索和自然语言处理中度量两个序列的相似度。

Question

输入两个字符串 s 和 t，返回将 s 转换为 t 所需的最少编辑步数。

你可以在一个字符串中进行三种编辑操作：插入一个字符、删除一个字符、将字符替换为任意一个字符。

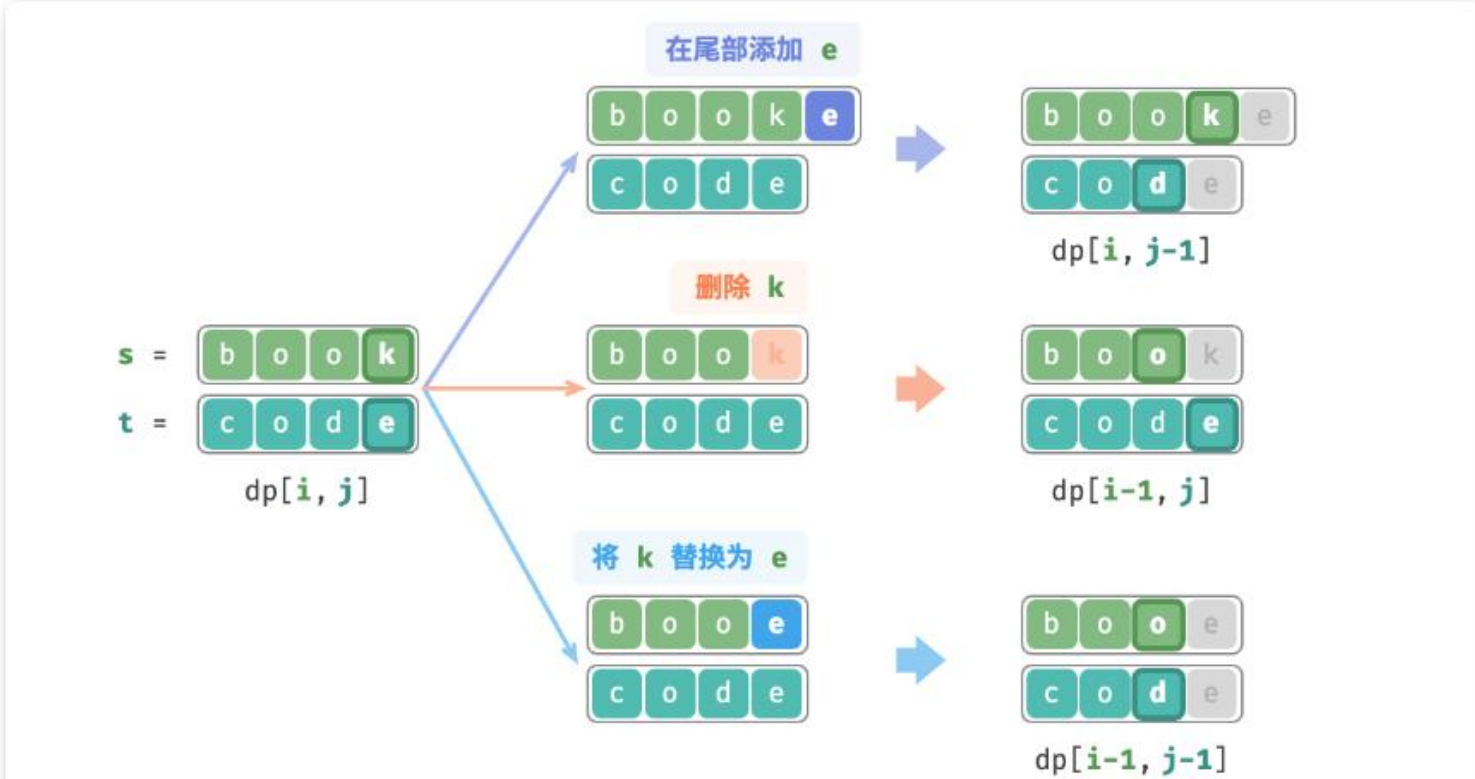
第二步：找出最优子结构，进而推导出状态转移方程

根据以上分析，可得最优子结构：$dp[i,j]$ 的最少编辑步数等于 $dp[i,j-1]$、$dp[i-1,j]$、$dp[i-1,j-1]$ 三者中的最少编辑步数，再加上本次的编辑步数 1。对应的状态转移方程为：

$$dp[i,j] = \min(dp[i,j-1], dp[i-1,j], dp[i-1,j-1]) + 1$$

请注意，**当 $s[i-1]$ 和 $t[j-1]$ 相同时，无须编辑当前字符**，这种情况下的状态转移方程为：

$$dp[i,j] = dp[i-1,j-1]$$







COOLMAKER
酷哥创客 人工智能教育

```cpp
#include<bits/stdc++.h>
using namespace std;
char a[2001], b[2001];
int k,dp[2001][2001];
int main()
{
    cin>>k>>a+1>>b+1;
    int lena = strlen(a+1);
    int lenb = strlen(b+1);
    for(int i = 0 ; i <= lena ; i++) dp[i][0] = i;
    for(int i = 0 ; i <= lenb ; i++) dp[0][i] = i;
    for(int i = 1 ; i <= lena ; i++)
    {
        for(int j = 1 ; j <= lenb ; j++)
        {
            if(a[i] == b[j])
            {
                dp[i][j] = dp[i-1][j-1] ;
            }
            else
            {
                dp[i][j]=min(dp[i-1][j], min(dp[i][j-1], dp[i-1][j-1]))+1;
            }
        }
    }
    cout<<(dp[lena][lenb]<= k ? "Yes" : "No")<<endl;
    return 0;
}
```

100分思路---极差优化DP

COOLMAKER
酷哥创客 人工智能教育

注意到 $k \leq 20$，不难想到如果 $|i-j|>k$ 一定不行，因为把它们变成长度一样的字符串都需要比 k 次操作更多。

所以我们可以改变状态，设 $f_{i,j}$ 表示 s 的前 i 个字符 和 t 的前 $i+j$ 个字符相等的最小操作次数。

然后转移方程是这个：

```
f[i][j]=min({f[i][j],f[i-1][j+1]+1,f[i][j-1]+1,f[i-1][j]+(s[i]!=t[r])});
```

分别代表删除，插入，改变，t_r 代表 t 的第 r 个字符。手推一下，结合上面的暴力，也容易得到。

通过映射的思想，可以把 j 的取值范围从 $-20 \leq j \leq 20$ 变成 $0 \leq j \leq 40$，当然，保险一些可以开的更大。

注意边界和一些特殊情况，代码也就没什么难的。

最后如果 $f_{n,m-n+p} \leq k$，其中 p 为映射时加的值，那么就有解，否则无解。

时间复杂度 $O(nk)$。







COOLMAKER
酷哥创客 人工智能教育

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
const int maxn = 500000 + 100;
int k, slen, tlen;
char s[maxn], t[maxn];
int dp[maxn][100];
int main()
{
    ios::sync_with_stdio(false);
    cin >> k >> (s + 1) >> (t + 1);
    slen = strlen(s + 1);
    tlen = strlen(t + 1);
    if(abs(slen - tlen) > k)
    {
        cout << "No" << endl;
        return 0;
    }
    memset(dp, 0x3f, sizeof(dp));
    dp[0][k] = 0;
    for(int i = 0; i <= slen; ++i)
    {
        for(int kk = -k; kk <= k; ++kk)
        {
            int j = i + kk;
            if(j < 0 || j > tlen)
            {
                continue;
            }
            int kkk = kk + k;
            if(i > 0 && kkk + 1 <= 2 * k)
            {
                dp[i][kkk] = min(dp[i][kkk], dp[i - 1][kkk + 1] + 1);
            }
            if(j > 0 && kkk - 1 >= 0)
            {
                dp[i][kkk] = min(dp[i][kkk], dp[i][kkk - 1] + 1);
            }
            if(i > 0 && j > 0)
            {
                dp[i][kkk] = min(dp[i][kkk], dp[i - 1][kkk] + (s[i] != t[j]));
            }
        }
    }
    cout << (dp[slen][k + tlen - slen] <= k ? "Yes" : "No") << endl;
    return 0;
}
```

2494: Emiya 家今天的饭

题目描述

Emiya 是个擅长做菜的高中生，他共掌握 n 种**烹饪方法**，且会使用 m 种**主要食材**做菜。为了方便叙述，我们对烹饪方法从 $1 \sim n$ 编号，对主要食材从 $1 \sim m$ 编号。

Emiya 做的每道菜都将使用**恰好一种**烹饪方法与**恰好一种**主要食材。更具体地，Emiya 会做 $a_{i,j}$ 道不同的使用烹饪方法 i 和主要食材 j 的菜（$1 \leq i \leq n$、$1 \leq j \leq m$），这也意味着 Emiya 总共会做 $\sum_{i=1}^{n} \sum_{j=1}^{m} a_{i,j}$ 道不同的菜。

Emiya 今天要准备一桌饭招待 Yazid 和 Rin 这对好朋友，然而三个人对菜的搭配有不同的要求，更具体地，对于一种包含 k 道菜的搭配方案而言：

- Emiya 不会让大家饿肚子，所以将做**至少一道菜**，即 $k \geq 1$
- Rin 希望品尝不同烹饪方法做出的菜，因此她要求每道菜的**烹饪方法互不相同**
- Yazid 不希望品尝太多同一食材做出的菜，因此他要求每种**主要食材**至多在**一半**的菜（即 $\lfloor \frac{k}{2} \rfloor$ 道菜）中被使用

这里的 $\lfloor x \rfloor$ 为下取整函数，表示不超过 x 的最大整数。

这些要求难不倒 Emiya，但他想知道共有多少种不同的符合要求的搭配方案。两种方案不同，当且仅当存在至少一道菜在一种方案中出现，而不在另一种方案中出现。

Emiya 找到了你，请你帮他计算，你只需要告诉他符合所有要求的搭配方案数对质数 $998,244,353$ 取模的结果。

输入

第 1 行两个用单个空格隔开的整数 n, m。

第 2 行至第 $n+1$ 行，每行 m 个用单个空格隔开的整数，其中第 $i+1$ 行的 m 个数依次为 $a_{i,1}, a_{i,2}, \cdots, a_{i,m}$。

输出

仅一行一个整数，表示所求方案数对 $998,244,353$ 取模的结果。

2494: Emiya 家今天的饭

COOLMAKER
酷哥创客 人工智能教育

样例输入1 复制

```
2 3
1 0 1
0 1 1
```

样例输出1 复制

```
3
```

样例输入2 复制

```
3 3
1 2 3
4 5 0
6 0 0
```

样例输出2 复制

```
190
```

【样例 1 解释】

由于在这个样例中，对于每组 i, j ，Emiya 都最多只会做一道菜，因此我们直接通过给出烹饪方法、主要食材的编号来描述一道菜。

符合要求的方案包括：

做一道用烹饪方法 1、主要食材 1 的菜和一道用烹饪方法 2、主要食材 2 的菜
做一道用烹饪方法 1、主要食材 1 的菜和一道用烹饪方法 2、主要食材 3 的菜
做一道用烹饪方法 1、主要食材 3 的菜和一道用烹饪方法 2、主要食材 2 的菜

因此输出结果为 $3 \ mod \ 998,244,353 = 3$ 。需要注意的是，所有只包含一道菜的方案都是不符合要求的，因为唯一的主要食材在超过一半的菜中出现，这不满足 Yazid 的要求。

【样例 2 解释】

Emiya 必须至少做 2 道菜。

做 2 道菜的符合要求的方案数为 100。

做 3 道菜的符合要求的方案数为 90。

因此符合要求的方案数为 100 + 90 = 190。

84分思路

COOLMAKER
酷哥创客 人工智能教育

首先考虑列的限制，发现若有不合法的列，则必然有且只有一列是不合法的：因为不可能有不同的两列数量都超过总数的一半。

于是发现列的限制容易容斥计算：每行选不超过一个的方案数 - 每行选不超过一个，且某一列选了超过一半的方案数。

总方案数

首先，计算所有满足前两个条件的方案数（即至少一道菜且烹饪方法互不相同）。对于每一行（烹饪方法），可以选择不选，或者选其中一种食材的菜。因此，对于第 i 行，方案数为 $(1 + \sum_{j=1}^{m} a_{i,j})$（加 1 表示不选）。所有行的方案数相乘后减去 1（减去全不选的情况），即：

$$\text{ans2} = \prod_{i=1}^{n} \left(1 + \sum_{j=1}^{m} a_{i,j}\right) - 1$$

84分思路

COOLMAKER 哥创客 人工智能教育

首先考虑列的限制，发现若有不合法的列，则必然有且只有一列是不合法的：因为不可能有不同的两列数量都超过总数的一半。

于是发现列的限制容易容斥计算：每行选不超过一个的方案数 - 每行选不超过一个，且某一列选了超过一半的方案数。

不合法方案数

那么考虑枚举不合法的一列。假设我们已经枚举了不合法的列为col，接下来会发现我们只关心一个数的位置是否在当前列；如果属于在其他列的情况，那么它具体在哪一列对当前列的合法性并无影响，我们并不需要考虑。

接下来设计状态。$f_{i,j,k}$表示对于col这一列，前i行在col列中选了j个，在其他列中选了k个，那么令s_i为第i行的总和，则有转移：

$$f_{i,j,k} = f_{i-1,j,k} + a_{i,col} * f_{i-1,j-1,k} + (s_i - a_{i,col}) * f_{i-1,j,k-1}$$

状态数$O(n^3)$，转移$O(1)$，算上枚举col，这一步复杂度是$O(mn^3)$的。统计如下和式的值并对每一列求和即可得到不合法的方案数：

$$\sum_{j>k} f_{n,j,k}$$

COOLMAKER
酷哥创客 人工智能教育

```cpp
#include<bits/stdc++.h>
using namespace std;
long long ans1=0;
for(int j=1; j<=m; j++) //考虑第j种食材
{
    // 初始化DP数组
    for(int i = 0; i <= n; i++)
        for(int k = 0; k <= n; k++)
            for(int k2 = 0; k2 <= n; k2++)
                dp[i][k][k2] = 0;
    dp[0][0][0] = 1;  // 初始状态
    // 四层循环暴力DP
    for(int i = 1; i <= n; i++) //考虑前i个烹饪方法
    {
        for(int k = 0; k <= i; k++) //使用了k个烹饪方法和i号食材制作菜
        {
            for(int k2 = 0; k2 <= i; k2++) //其他菜用了k2个烹饪方法
            {
                //                    不用          用i烹j              不用i烹j
                // 不选当前行
                dp[i][k][k2] = dp[i-1][k][k2];
                // 选食材j
                if(k > 0)
                {
                    dp[i][k][k2] = (dp[i][k][k2] + dp[i-1][k-1][k2] * a[i][j]) % mod;
                }
                // 选其他食材
                if(k2 > 0)
                {
                    dp[i][k][k2] = (dp[i][k][k2] + dp[i-1][k][k2-1] * sum[i][j]) % mod;
                }
            }
        }
    }
    // 统计不合法方案：食材j使用次数 > 其他食材总使用次数
    for(int k = 1; k <= n; k++)
    {
        for(int k2 = 0; k2 < k; k2++) //不合法的中只会有1种食材不合法，k>k2，j号食材用多了
        {
            ans1 = (ans1 + dp[n][k][k2]) % mod;
        }
    }
}
    cout<< (ans2 - ans1 + mod) % mod << "\n";
    return 0;
}
```

100分思路---极差优化DP

COOLMAKER
酷哥创客 人工智能教育

考虑进一步优化，剪去无用状态：注意到在不合法情况的计算过程中，也就是$f_{i,j,k}$的转移过程中，我们实际上并不关心j,k的具体数值，而只关心相对的大小关系；所以我们可以将状态变为$f_{i,j}$，表示前i行，当前列的数比其他列的数多了j个，则有转移：

$$f_{i,j} = f_{i-1,j} + a_{i,col} * f_{i-1,j-1} + (s_i - a_{i,col}) * f_{i-1,j+1}$$

转移仍然是$O(1)$的，但总复杂度降为$O(mn^2)$，可以通过此题。

```cpp
#include<bits/stdc++.h>
using namespace std;
const long long mod=998244353;
int n, m;
long long a[105][2005], sum[105][2005], dp[105][105][105];
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin>>n>>m;
    for(int i=1; i<=n; i++)
    {
        for(int j=1; j<=m; j++)
        {
            cin>>a[i][j];
            sum[i][0]+=a[i][j];
            sum[i][0]%=mod;
        }
    }
    for(int i=1; i<=n; i++)
    {
        for(int j=1; j<=m; j++)
        {
            sum[i][j]=(sum[i][0]-a[i][j]+mod)%mod;
        }
    }
    long long ans1=0;
    for(int j=1; j<=m; j++) //考虑第j种食材
    {...}
    // 计算总方案数（至少一道菜）
    long long ans2 = 1;
    for(int i = 1; i <= n; i++)
    {
        ans2 = (ans2 * (sum[i][0] + 1)) % mod;  // +1表示可以不选该行
    }
    ans2 = (ans2 - 1 + mod) % mod;  // 减去全不选的情况
    //合法=总方案-不合法
    cout<< (ans2 - ans1 + mod) % mod <<"\n";
    return 0;
}
```

```cpp
#include<bits/stdc++.h>
using namespace std;
const long long mod=998244353;
int n, m;
long long a[105][2005], sum[105][2005], dp[105][205];
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin>>n>>m;
    for(int i=1; i<=n; i++)
    {
        for(int j=1; j<=m; j++)
        {
            cin>>a[i][j];
            sum[i][0]+=a[i][j];
            sum[i][0]%=mod;
        }
    }
    for(int i=1; i<=n; i++)
    {
        for(int j=1; j<=m; j++)
        {
            sum[i][j]=(sum[i][0]-a[i][j]+mod)%mod;
        }
    }
    long long ans1=0;
    for(int j=1; j<=m; j++) //考虑第j种食材
    {...}
    // 计算总方案数（至少一道菜）
    long long ans2 = 1;
    for(int i = 1; i <= n; i++)
    {
        ans2 = (ans2 * (sum[i][0] + 1)) % mod;  // +1表示可以不选该行
    }
    ans2 = (ans2 - 1 + mod) % mod;  // 减去全不选的情况
    //合法=总方案-不合法
    cout<< (ans2 - ans1 + mod) % mod <<"\n";
    return 0;
}
```

COOLMAKER
酷哥创客 人工智能教育

```cpp
    long long ans1=0;
    for(int j=1; j<=m; j++) //考虑第j种食材
    {
        // 初始化DP数组
        for(int i = 0; i <= n; i++)
            for(int k = 0; k <= n; k++)
                for(int k2 = 0; k2 <= n; k2++)
                    dp[i][k][k2] = 0;
        dp[0][0][0] = 1;  // 初始状态
        // 四层循环暴力DP
        for(int i = 1; i <= n; i++) //考虑前i个烹饪方法
        {
            for(int k = 0; k <= i; k++) //使用了k个烹饪方法和i号食材制作菜
            {
                for(int k2 = 0; k2 <= i; k2++) //其他菜用了k2个烹饪方法
                {
                    //                    不用          用i烹j                不用i烹j
                    // 不选当前行
                    dp[i][k][k2] = dp[i-1][k][k2];
                    // 选食材j
                    if(k > 0)
                    {
                        dp[i][k][k2] = (dp[i][k][k2] + dp[i-1][k-1][k2] * a[i][j]) % mod;
                    }
                    // 选其他食材
                    if(k2 > 0)
                    {
                        dp[i][k][k2] = (dp[i][k][k2] + dp[i-1][k][k2-1] * sum[i][j]) % mod;
                    }
                }
            }
        }
        // 统计不合法方案：食材j使用次数 > 其他食材总使用次数
        for(int k = 1; k <= n; k++)
        {
            for(int k2 = 0; k2 < k; k2++) //不合法的中只会有1种食材不合法，k>k2，j号食材用多了
            {
                ans1 = (ans1 + dp[n][k][k2]) % mod;
            }
        }
    }
```

```cpp
26      long long ans1=0;
27      for(int j=1; j<=m; j++) //考虑第j种食材
28      {
29          // 初始化DP数组
30          for(int i = 0; i <= n; i++)
31              for(int k = 0; k <= 2*n; k++)
32                  dp[i][k]= 0;  // 初始状态
33          dp[0][n]= 1;  // 初始状态
34          for(int i = 1; i <= n; i++) //考虑前i个烹饪方法
35          {
36              for(int k = 0; k <= 2*n; k++) //用来做j和用来其他的之差
37              {
38                  //                    不用          用i烹j                不用i烹j
39                  // 不选当前行
40                  dp[i][k] = dp[i-1][k];
41                  // 选食材j
42                  if(k > 0)
43                  {
44                      dp[i][k] = (dp[i][k] + dp[i-1][k-1] * a[i][j]) % mod;
45                  }
46                  // 选其他食材
47                  if(k < 2*n)
48                  {
49                      dp[i][k] = (dp[i][k]+ dp[i-1][k+1] * sum[i][j]) % mod;
50                  }
51              }
52          }
53          // 统计不合法方案：食材j使用次数 > 其他食材总使用次数
54          //不合法的中只会有1种食材不合法，枚举这种食材进行DP，k>k2，j号食材用多了
55          for(int k = n+1; k <= 2*n; k++)
56          {
57              ans1 = (ans1 + dp[n][k]) % mod;
58          }
59      }
```



根号分治优化



星球跳跃

COOLMAKER
人工智能教育

题目描述

小酷宝驾驶着他的宇宙飞船在银河系中探索，发现了一个由 N 个星球组成的星际网络。每个星球 i 都有一个独特的空间跳跃引擎，可以产生 A_i 光年的跳跃距离。

初始时：

只有星球 1 被标记为已探索

其他 $N-1$ 个星球都未被探索

小酷宝的飞船停在星球 1 上

小酷宝可以进行以下操作任意次数：

> 1. *当飞船在星球 i 时，选择一个正整数 x*
> 2. *跳跃到星球 $i + A_i \times x$ （不能超过 N 号星球）*
> 3. *将目标星球标记为已探索*

问：探索结束时，所有可能的已探索星球组合有多少种？由于答案可能很大，请对 998244353 取模。

输入

第一行：星球总数 N

第二行： N 个整数，表示每个星球的跳跃距离 A_1 到 A_N

$1 \leq N \leq 2 \times 10^5$

$1 \leq A_i \leq 2 \times 10^5$

输出

输出可能的探索组合数对 998244353 取模的结果

> 形式化题意：
>
> 给定 n 和 $a_1 \dots a_n$。求有多少个正整数序列 $b_1 \dots b_k$（长度 k 任意），满足：
>
> - 对于所有 $i \in [1, k)$，有 $b_i < b_{i+1} \le n$；
> - $b_1 = 1$；
> - 对于所有 $i \in (1, k]$，存在正整数 x，使得 $b_i = b_{i-1} + a_{b_{i-1}} \cdot x$。

首先对「存在正整数 x，使得 $b_i = b_{i-1} + a_{b_{i-1}} \times x$」变形：

$$
\begin{aligned}
b_i &= b_{i-1} + a_{b_{i-1}} \cdot x \\
b_i - b_{i-1} &= a_{b_{i-1}} \cdot x \\
x &= \frac{b_i - b_{i-1}}{a_{b_{i-1}}}
\end{aligned}
$$

也就是说需要保证 $a_{b_{i-1}} \mid (b_i - b_{i-1})$。

COOLMAKER
酷哥创客 人工智能教育

然后直接 DP。设 f_i 表示 $b_k = i$ 的 b 的数量。

转移显然。$\mathcal{O}(n^2)$ 的初始 DP：

```
f[1] = 1;
for (int i = 2; i <= n; ++ i )
    for (int j = 1; j < i; ++ j )
        if ((i - j) % a[j] == 0)
            (f[i] += f[j]) %= P;
```

答案为 $\sum_{i=1}^{n} f_i$。

改成刷表，枚举 a_i 的所有倍数：

```
f[1] = 1;
for (int i = 1; i <= n; ++ i )
    for (int j = a[i]; j + i <= n; j += a[i])
        (f[j + i] += f[i]) %= P;
```

继续

```
f[1] = 1;
for (int i = 1; i <= n; ++ i )
    for (int j = a[i] + i; j <= n; j += a[i])
        (f[j] += f[i]) %= P;
```

可以发现，当 a_i 很大时，内层循环的复杂度是很低的。但问题就出在了 a_i 比较小的时候。

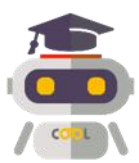
直接联想根号分治。令 $B = \sqrt{n}$。

当 $a_i > B$ 时，还是上面那么做。否则维护额外信息。

首先把 $(i - j) \bmod a_j = 0$ 替换为 $i \equiv j \pmod{a_j}$。

理解 DP 可以从集合角度分析。那么我们将 $s_{x,y}$ 定义为：

- $s_{x,y}$ 表示所有满足 $x = a_i, y = i \bmod a_i$ 的 i 构成的集合。即 $s_{x,y} = \{i \mid x = a_i, y = i \bmod a_i\}$。



并且定义 $g(A)=\sum_{i\in A} f_i$，其中 A 是一个集合。

转移有：

$$f_k=\sum_{i=1}^{B} g(s_{i,k \bmod i})$$

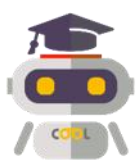
这样理解转移方程：

- 首先令 $s_{i,k \bmod i}$ 这个集合中的一个元素为 t，即 $t\in s_{i,k \bmod i}$，那么**根据定义**有：
  - $i=a_t$；
  - $k \bmod i=t \bmod a_t$；

  稍微解释一下：这里 i 表示定义中的 x，$k \bmod i$ 表示定义中的 y，t 表示定义中的 i，a_t 表示定义中的 a_i。事实上就是这八个东西互相替换了。

- 那么将 1 式代入 2 式，有：
  - $k \bmod a_t=t \bmod a_t$，即 $k\equiv t \pmod{a_t}$。

然后发现这样就满足了最开始 f_k 的定义。







COOLMAKER
酷哥创客 人工智能教育

```cpp
#include<bits/stdc++.h>
using ll = long long;
const ll mod = 998244353;
using namespace std;
ll n, sqr=700, ans, a[200005], sum[1005][1005], f[200005];
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n;
    for(int i=1; i<=n; i++) cin>>a[i];
    f[1]=1;
    for(int i=1; i<=n; i++)
    {
        for(int j=1; j<=sqr; j++) f[i]+=sum[j][i%j], f[i]%=mod;
        if(a[i]<=sqr)
        {
            sum[a[i]][i%a[i]]+=f[i], sum[a[i]][i%a[i]]%=mod;
        }
        else
        {
            for(int j=i+a[i]; j<=n; j+=a[i]) f[j]+=f[i], f[j]%=mod;
        }
        ans+=f[i], ans%=mod;
    }
    cout<<ans<<"\n";
    return 0;
}
```



最值相关的优化





收集糖果

★★★★☆

题目描述

在一个慵懒的周末午后，小酷宝抱着一桶肯德基炸鸡，手里还拿着麦当劳的巨无霸，正准备大快朵颐。就在这时，他突然打了个盹，进入了梦乡。梦中，小酷宝变成了一个热爱学习的小学生，他总是能够迅速完成作业。一天，他早早地完成了所有的家庭作业，感到有些无聊。他的两个好朋友，小束和小马，也正好完成了作业，于是他们决定一起找点游戏玩。

小酷宝提议："我们来玩一个收集糖果的游戏吧，我家里刚好有一些特别的糖果。"小束和小马都表示同意，于是他们围坐在小酷宝家的客厅里。小酷宝拿出了一些特别的糖果，这些糖果有 n 颗，每颗糖果有一个颜色 F 和甜度 D ,颜色不超过 k 种。他们决定玩一种新的糖果收集游戏，规则如下：

1.将一颗糖果依次放入一篮子糖果的末尾，如果放入之前这篮子糖果中已有与这颗糖果颜色相同的糖果，你可以选择将这颗糖果和**任意一颗颜色相同**的糖果之间的所有糖果全部取出篮子（包括这两颗糖果）。
2.取出的糖果将不再放回篮子，并且你将**得到与取出的所有糖果甜度和相等的分数。**

小酷宝想知道，对于给定顺序的一篮子糖果，他能够获得的分数最多是多少？输入顺序即为小酷宝放入篮子的顺序，即 F_i 表示第 i 颗放入的糖果的颜色， D_i 表示第 i 颗放入的糖果的甜度。

输入

第一行两个整数 n, k ，分别表示糖果的数量和颜色的种类数。
第二行， n 个整数，第 i 个整数 F_i 表示第 i 颗糖果的颜色，满足 $1 \leq F_i \leq k$ 。
第三行， n 个整数，第 i 个整数 D_i ;表示第 i 颗糖果的甜度。
数据规模与约定
对于 20% 的数据， $1 \leq n, k \leq 15$
对于 40% 的数据， $1 \leq n, k \leq 3000$
对于 60% 的数据， $1 \leq n, k \leq 3000, 1 \leq D_i \leq 10^9$
对于 100% 的数据， $1 \leq n, k \leq 10^6, 1 \leq F_i \leq k, 1 \leq D_i \leq 10^9$

输出

输出一个整数，表示小酷宝能够获得的最大分数。

样例输入1 复制

```
7 3
1 2 1 2 3 2 3
1 2 1 2 3 2 3
```

样例输出1 复制

```
13
```

样例一解释

1212323，依次取出红色部分和蓝色部分的糖果。

问题 C: 收集糖果

有 n 颗糖果，每颗糖果有一个颜色和一个甜度值。如果两颗糖果颜色相同就可以把两颗糖果间的糖果都拿光（包括这两个）并得到所有糖果的甜度值加和。不同的拿取顺序和策略，会有不同的和，求最大可能的甜度值之和。

解题思路---前缀和、DP以及优化

暴力思路：

这是一道很明显的线性动态规划题，取出一段区间的要求是这段区间的两个端点的颜色是一样的。

定义 dp[i] 表示前 i 颗糖果处理后能够得到的最大得分。

对于第 i 颗糖果有计分/不计分两种情况：

- 不计分则分数不变：dp[i-1]。
- 计分则需要考虑要和哪个相同颜色糖果进行合并取出，要的是最大的那一种合并：
  即 $\max_{j\in\text{同色位置}}(dp[j-1]+(sum[i]-sum[j-1]))$，$sum$ 前缀和数组存了前 i 颗糖果的甜度和。

$$dp[i]=\max(dp[i-1],\quad \max_{j\in\text{同色位置}}(dp[j-1]+(sum[i]-sum[j-1])))$$

时间复杂度为 $\mathcal{O}(n^2)$，绝对超时。

问题 C: 收集糖果

有 n 颗糖果，每颗糖果有一个颜色和一个甜度值。如果两颗糖果颜色相同就可以把两颗糖果间的糖果都拿光（包括这两个）并得到所有糖果的甜度值加和。不同的拿取顺序和策略，会有不同的和，求最大可能的甜度值之和。

解题思路---前缀和、DP以及优化

暴力思路:

这是一道很明显的线性动态规划题，取出一段区间的要求是这段区间的两个端点的颜色是一样的。

定义 dp[i] 表示前 i 颗糖果处理后能够得到的最大得分。

对于第 i 颗糖果有计分/不计分两种情况：

- 不计分则分数不变：dp[i-1]。
- 计分则需要考虑要和哪个相同颜色糖果进行合并取出，要的是最大的那一种合并：
  即 $\max_{j\in同色位置}(dp[j-1]+(sum[i]-sum[j-1]))$，$sum$ 前缀和数组存了前 i 颗糖果的甜度和。

$$dp[i]=\max(dp[i-1],\quad \max_{j\in同色位置}(dp[j-1]+(sum[i]-sum[j-1])))$$

时间复杂度为 $\mathcal{O}(n^2)$，绝对超时。

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N=1e6+5;
typedef long long ll;
int n, k;
ll f[N], d[N], dp[N], sum[N], mx[N], fst[N];
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n>>k;
    for(int i=1; i<=n; i++)cin>>f[i];
    for(int i=1; i<=n; i++)
    {
        cin>>d[i];
        sum[i]=sum[i-1]+d[i];
    }
    for(int i=1; i<=n; i++)
    {
        dp[i]=dp[i-1]; //不计分
        for(int j=1; j<i; j++)
        {
            if(f[i]==f[j]) // 是一个同色位置，尝试合并计算新得分，保留最大得分
            {
                dp[i]=max(dp[i], dp[j-1]+sum[i]-sum[j-1]);
            }
        }
    }
    cout<<dp[n]<<"\n";
    return 0;
}
```

思路优化：

对于计分的情况，$\max_{j\in\text{同色位置}}(dp[j-1]+(sum[i]-sum[j-1]))$，只会从最大的合并中进行转移。

可以发现 $\max_{j\in\text{同色位置}}(dp[j-1]-sum[j-1])+sum[i]$，也就是无论怎么合并，只和 $dp[j-1]-sum[j-1]$ 的值有关，而与 i 无关。

所以我们只需要记录每种颜色的糖果最大的 $dp[j-1]-sum[j-1]$ 的值即可，每次转移就不需要枚举前 $i-1$ 糖果了。

时间复杂度为 $\mathcal{O}(n)$。

思路优化：

对于计分的情况，$\max_{j\in同色位置}(dp[j-1]+(sum[i]-sum[j-1]))$，只会从最大的合并中进行转移。

可以发现 $\max_{j\in同色位置}(dp[j-1]-sum[j-1])+sum[i]$，也就是无论怎么合并，只和 $dp[j-1]-sum[j-1]$ 的值有关，而与 i 无关。

所以我们只需要记录每种颜色的糖果最大的 $dp[j-1]-sum[j-1]$ 的值即可，每次转移就不需要枚举前 $i-1$ 糖果了。

时间复杂度为 $\mathcal{O}(n)$。

标准程序---时间复杂度$\mathcal{O}(n)$

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N=1e6+5;
typedef long long ll;
int n, k;
ll f[N], d[N], dp[N], sum[N], mx[N], fst[N];
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n>>k;
    for(int i=1; i<=n; i++)cin>>f[i];
    for(int i=1; i<=n; i++)
    {
        cin>>d[i];
        sum[i]=sum[i-1]+d[i];
    }
    memset(mx, -0x3f, sizeof(mx));
    for(int i=1; i<=n; i++)
    {
        //不计分  和 计分 中选最大的
        dp[i]=max(dp[i-1], mx[f[i]]+sum[i]);

        //记录当前颜色最大的 dp[i-1]+sum[i-1]值，优化计分时的转移
        mx[f[i]]=max(mx[f[i]], dp[i-1]-sum[i-1]);
    }
    cout<<dp[n];
    return 0;
}
```

4646: 染色（color）

题目描述

给定一个长度为 n 的正整数数组 A，其中所有数从左至右排成一排。

你需要将 A 中的每个数染成红色或蓝色之一，然后按如下方式计算最终得分：

设 C 为长度为 n 的整数数组，对于 A 中的每个数 A_i（$1 \le i \le n$）：

如果 A_i 左侧没有与其同色的数，则令 $C_i = 0$。

否则，记其左侧**与其最靠近的同色数**为 A_j，若 $A_i = A_j$，则令 $C_i = A_i$，否则令 $C_i = 0$。

你的最终得分为 C 中所有整数的和，即 $\sum_{i=1}^{n} C_i$。你需要最大化最终得分，请求出最终得分的最大值。

输入

本题有多组测试数据。

输入的第一行包含一个正整数 T，表示数据组数。

接下来包含 T 组数据，每组数据的格式如下：

第一行包含一个正整数 n，表示数组长度。

第二行包含 n 个正整数 $A_1, A_2, \cdots, A_n$，表示数组 A 中的元素。

对于所有测试数据，保证：$1 \le T \le 10$，$2 \le n \le 2 \times 10^5$，$1 \le A_i \le 10^6$。

测试点	n	A_i
$1 \sim 4$	≤ 15	≤ 15
$5 \sim 7$	$\le 10^2$	$\le 10^2$
$8 \sim 10$	$\le 2,000$	$\le 2,000$
$11, 12$	$\le 2 \times 10^4$	$\le 10^6$
$13 \sim 15$	$\le 2 \times 10^5$	≤ 10
$16 \sim 20$		$\le 10^6$

输出

对于每组数据：输出一行包含一个非负整数，表示最终得分的最大可能值。

样例输入1 复制

```
3
3
1 2 1
4
1 2 3 4
8
3 5 2 5 1 2 1 4
```

样例输出1 复制

```
1
0
8
```

提示

【样例 1 解释】

1.将 A_1, A_2 染成红色，将 A_3 染成蓝色（121），其得分计算方式如下：

对于 A_1，由于其左侧没有红色的数，所以 $C_1 = 0$。

对于 A_2，其左侧与其最靠近的红色数为 A_1。由于 $A_1 \neq A_2$，所以 $C_2 = 0$。

对于 A_3，由于其左侧没有蓝色的数，所以 $C_3 = 0$。

该方案最终得分为 $C_1 + C_2 + C_3 = 0$。

2.将 A_1, A_2, A_3 全部染成红色（121），其得分计算方式如下：

对于 A_1，由于其左侧没有红色的数，所以 $C_1 = 0$。

对于 A_2，其左侧与其最靠近的红色数为 A_1。由于 $A_1 \neq A_2$，所以 $C_2 = 0$。

对于 A_3，其左侧与其最靠近的红色数为 A_2。由于 $A_2 \neq A_3$，所以 $C_3 = 0$。

该方案最终得分为 $C_1 + C_2 + C_3 = 0$。

3.将 A_1, A_3 染成红色，将 A_2 染成蓝色（121），其得分计算方式如下：

对于 A_1，由于其左侧没有红色的数，所以 $C_1 = 0$。

对于 A_2，由于其左侧没有蓝色的数，所以 $C_2 = 0$。

对于 A_3，其左侧与其最靠近的红色数为 A_1。由于 $A_1 = A_3$，所以 $C_3 = A_3 = 1$。

该方案最终得分为 $C_1 + C_2 + C_3 = 1$。

可以证明，没有染色方案使得最终得分大于 1。

对于第二组数据，可以证明，任何染色方案的最终得分都是 0。

对于第三组数据，一种最优的染色方案为将 A_1, A_2, A_4, A_5, A_7 染为红色，将 A_3, A_6, A_8 染为蓝色（35251214），其对应 $C = [0, 0, 0, 5, 0, 2, 1, 0]$，最终得分为 8。

设 f[i][0/1] 代表和前一个位置同色/不同色的最大得分

如果和前面一个位置同颜色，那么得分为前一个位置的最大得分 max(f[i − 1][0], f[i − 1][1])，加上当前位置的得分，如果和前面数字相同，那么得分 a[i]，否则不得分 (a[i] == a[i − 1] ? a[i] : 0)

$$f[i][0] = \max(f[i-1][0], f[i-1][1]) + (a[i] == a[i-1] \ ? \ a[i] : 0);$$

定义 s[i] 代表 前 i 个元素都同色时的得分
如果和前面一个位置不同颜色，那么得分就是和前面所有位置都不同颜色的得分 s[i − 1] 与 考虑和前面 i-2个的哪个位置同色时的得分之间取最大值

$$f[i][1] = \begin{cases} s[i-1] & \\ max(f[i][1],\ f[j+1][1] + a[i] + s[i-1] - s[j+1]) & , j \le i-2 \ and \ a[i] = a[j] \\ max(f[i][1],\ f[j+1][1] + s[i-1] - s[j+1]) & , j \le i-2 \ and \ a[i] \ne a[j] \end{cases}$$

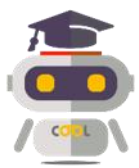
时间复杂度 $O(n^2)$，期望得分 50pts。

COOLMAKER
酷哥创客 人工智能教育

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 1e6 + 5;
int n, T;
long long a[N], f[N][2], mx[N];
long long s[N], ans;
int main()
{
    cin >> T;
    while (T--)
    {
        cin >> n;

        for (int i = 1; i <= n; i++) cin >> a[i];
        for (int i = 2; i <= n; i++) s[i] = (a[i] == a[i - 1] ? s[i - 1] + a[i] : s[i - 1]);
        memset(mx, -0x3f, sizeof(mx));
        memset(f, 0, sizeof(f));
        for (int i = 1; i <= n; i++)
        {
            f[i][0] = max(f[i - 1][0], f[i - 1][1]) + (a[i] == a[i - 1] ? a[i] : 0);//和前一个同色
            f[i][1] = s[i-1]; //和前一个不同色，先不和前面所有元素同色
            for(int j = 1; j <= i-2; j++) //考虑和前面i-2个元素的那个同色
            {
                if(a[i]==a[j])
                {
                    f[i][1] = max(f[i][1], f[j+1][1]+a[i]+s[i-1]-s[j+1]);

                }else
                {
                    f[i][1] = max(f[i][1], f[j+1][1]+s[i-1]-s[j+1]);
                }
            }
        }
        cout << max(f[n][0], f[n][1]) << endl;
    }
    return 0;
}
```



最值优化DP

$$f[i][1] = \begin{cases} s[i-1] & \\ max(f[i][1],\ f[j+1][1] + a[i] + s[i-1] - s[j+1]) & , j \le i-2\ and\ a[i] = a[j] \\ max(f[i][1],\ f[j+1][1] + s[i-1] - s[j+1]) & , j \le i-2\ and\ a[i] \ne a[j] \end{cases}$$

把中间的for循环优化掉，对于$j \le i-2\ and\ a[i] = a[j]$ ，可以发现无论怎么合并 $f[j+1][1] + a[i] - s[j+1]$ 的值时不会变的，而与 i 无关，我们可以使用一个数组去记录前 $i-2$ 个元素中和当前位置同数字大小的位置的 $f[j+1][1] + a[i] - s[j+1]$ 最大值。

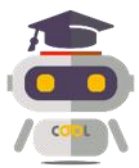
而对于 $j \le i-2\ and\ a[i]! = a[j]$ ，也是类似的，我们去定义一个变量 mxx 去存储 前 $i-2$ 个元素中最大的 $f[j+1][1] - s[j+1]$ 即可。

$$f[i][1] = \begin{cases} s[i-1] & \\ max(f[i][1], mx[a[i]] + s[i-1]) & \\ max(f[i][1], mxx + s[i-1]) & \end{cases}$$

时间复杂度 $O(n)$，期望得分 100pts。

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 1e6 + 5;
int n, T;
long long a[N], f[N][2], mx[N];
long long s[N], ans;
int main()
{
    cin >> T;
    while (T--)
    {
        cin >> n;
        for (int i = 1; i <= n; i++) cin >> a[i];
        for (int i = 2; i <= n; i++) s[i] = (a[i] == a[i - 1] ? s[i - 1] + a[i] : s[i - 1]);
        memset(mx, -0x3f, sizeof(mx));
        memset(f, 0, sizeof(f));
        long long mxx=0;
        for (int i = 1; i <= n; i++)
        {
            f[i][0] = max(f[i - 1][0], f[i - 1][1]) + (a[i] == a[i - 1] ? a[i] : 0);
            f[i][1] = s[i-1];
            if (mx[a[i]]!=-0x3f3f3f3f3f3f3f3f) f[i][1] = max(f[i][1], mx[a[i]] + s[i-1]);
            if(i>2)f[i][1] = max(f[i][1], mxx + s[i-1]);

            mx[a[i-1]] = max(mx[a[i-1]], f[i][1] + a[i-1] - s[i]);
            mxx = max(mxx, f[i][1] - s[i]);
        }
        cout << max(f[n][0], f[n][1]) << endl;
    }
    return 0;
}
```



3299: 修剪草坪

COOLMAKER
酷哥创客 人工智能教育

题目描述

在一年前赢得了小镇的最佳草坪比赛后，FJ 变得很懒，再也没有修剪过草坪。现在，新一轮的最佳草坪比赛又开始了，FJ 希望能够再次夺冠。

然而，FJ 的草坪非常脏乱，因此，FJ 只能够让他的奶牛来完成这项工作。FJ 有 N 只排成一排的奶牛，编号为 1 到 N。每只奶牛的效率是不同的，奶牛 i 的效率为 E_i。

靠近的奶牛们很熟悉，如果 FJ 安排超过 K 只连续的奶牛，那么这些奶牛就会罢工去开派对。因此，现在 FJ 需要你的帮助，计算 FJ 可以得到的最大效率，并且该方案中没有连续的超过 K 只奶牛。

输入

第一行：空格隔开的两个整数 N 和 K；

第二到 $N+1$ 行：第 $i+1$ 行有一个整数 E_i。

对于全部数据，$1 \leq N \leq 10^5, 0 \leq E_i \leq 10^9$。

输出

一行一个值，表示 FJ 可以得到的最大的效率值。

样例输入

```
5 2
1
2
3
4
5
```

样例输出

```
12
```

提示

FJ 有 5 只奶牛，他们的效率为 $1, 2, 3, 4, 5$。他们希望选取效率总和最大的奶牛，但是他不能选取超过 2 只连续的奶牛。FJ 选择出了第三只以外的其他奶牛，总的效率为 $1+2+4+5=12$。

令 $dp[i][1]$ 表示在前 i 头奶牛中,选了第 i 头奶牛能获得的最大效率

$dp[i][0]$ 表示在前 i 头奶牛中,不选第 i 头奶牛能获得的最大效率

显然可以得出以下转移方程 (**注意 K 是题目给出的区间长度**)

$$dp[i][0] = max(dp[i-1][0], dp[i-1][1])$$

$$dp[i][1] = max\{dp[j][0] + \sum_{j=i-K+1}^{i} E_j\}$$

可以预处理出前缀和优化掉一重循环,转移方程就变成了如下所示

$$dp[i][1] = max\{dp[j][0] + Sum[i] - Sum[j]\}$$

因为最终都要加上一个 $Sum[i]$,所以可以把 $Sum[i]$ 移到 max 函数外,所以

$$dp[i][1] = max\{dp[j][0] - Sum[j]\} + Sum[i] \quad (i - K \le j < i)$$

这时候我们可以发现,要使 $dp[i][1]$ 最大,我们就要让 $dp[j][0] - Sum[j]$ 尽可能的大,所以只需要用一个单调队列维护长度为 K 的区间中 $dp[j][0] - Sum[j]$ 的最大值就好了.

令 $dp[i]$ 表示到前 i 头奶牛为止能得到的最大效率

我们可以在区间 $[i-K,i]$ 间枚举休息的奶牛,所以转移方程就可以初步推出

$$dp[i]=max\{dp[j-1]+\sum_{k=j+1}^{i}E_k\}\quad(i-K\le j\le i)$$

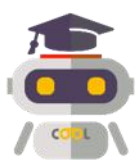
同样,我们预处理出前缀和进行优化

$$dp[i]=max\{dp[j-1]+Sum[i]-Sum[j]\}\quad(i-K\le j\le i)$$

然后也是把 $Sum[i]$ 移到外面

$$dp[i]=max\{dp[j-1]-Sum[j]\}+Sum[i]\quad(i-K\le j\le i)$$

我们发现,这里又可以用一个单调队列维护 $dp[j-1]-Sum[j]$,问题也就迎刃而解了.



单调队列做法

COOLMAKER 酷哥创客 人工智能教育

令 $dp[i]$ 表示到前 i 头奶牛为止能得到的最大效率

我们可以在区间 $[i-K,i]$ 间枚举休息的奶牛,所以转移方程就可以初步推出

$$dp[i]=max\{dp[j-1]+\sum_{k=j+1}^{i}E_k\}\quad(i-K\le j\le i)$$

同样,我们预处理出前缀和进行优化

$$dp[i]=max\{dp[j-1]+Sum[i]-Sum[j]\}\quad(i-K\le j\le i)$$

然后也是把 $Sum[i]$ 移到外面

$$dp[i]=max\{dp[j-1]-Sum[j]\}+Sum[i]\quad(i-K\le j\le i)$$

我们发现,这里又可以用一个单调队列维护 $dp[j-1]-Sum[j]$,问题也就迎刃而解了.

```cpp
#include<bits/stdc++.h>
using namespace std;
```

```cpp
#include<bits/stdc++.h>
using namespace std;
int n,k;
long long E[100010],sum[100010],dp[100010];
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n>>k;
    for(int i=1; i<=n; i++)
    {
        cin>>E[i],sum[i]=sum[i-1]+E[i];
    }
    for(int i=1; i<=n; i++)
    {
        for(int j=max(0,i-k);j<=i;j++)
        {
            dp[i]=max(dp[i],dp[j-1]+sum[i]-sum[j]);
        }
    }
    cout<<dp[n];
}
```

```cpp

        dp[i]=que[q[head]]+sum[i];
    }
    cout<<dp[n];
}
```

5468: 宝石收藏

题目描述

小酷宝在神秘洞穴中发现了 N 颗闪耀的宝石，整齐地排列在石台上。每颗宝石 i （$1 \leq i \leq N$）都有一个独特的能量值 e_i 和价值 w_i 。所有宝石的能量值都互不相同。

小酷宝想挑选一些宝石带回家，但必须满足条件：按照挑选顺序排列时，宝石的能量值必须严格递增

请帮助小酷宝计算他能带走的宝石的最大总价值。

输入

第一行有一个整数 N ,代表宝石的数量

第二行有 N 个整数 e_i ,代表对应宝石的能量值

第三行有 N 个整数 w_i ,代表对应宝石的价值

$1 \leq N \leq 2 \times 10^5$

$1 \leq e_i \leq N$ （且各 e_i 互不相同）

$1 \leq w_i \leq 10^9$

样例输入1 复制

```
4
3 1 4 2
10 20 30 40
```

样例输出1 复制

```
60
```

选择第2和第4颗宝石，能量序列为1→2，总价值20+40=60

输出

输出宝石的最大总价值。

令 f_i 为前 i 个花满足题目“高度单调递增”的条件，花权值之和的最大值。易得转移方程为：

$$f_i = \max_{j=1}^{i-1}\{f_j \mid h_j < h_i\} + a_i$$

由于 n 到 2×10^5，n^2 的 dp 会炸，需要优化。

转到值域上开一个线段树维护区间最大值即可。

```cpp
#include <b
using names
using ll =
int n, h[20
ll dp, ans;
struct SegT
{...}
int main()
{
    ios::sy
    cin>>n;
    for(int
    for(int
    for(int
    {
        if(
        els
        tre
        ans
    }
    cout<<a
    return
}
```

```cpp
struct SegTree
{
    ll a[200010<<2];
    void pushup(int rt)
    {
        a[rt]=max(a[rt<<1], a[rt<<1|1]);
    }
    void modify(int pos, ll k, int ql, int qr, int rt)
    {
        if(ql==qr)
        {
            a[rt]=max(a[rt],k);
            return;
        }
        int mid=(ql+qr)>>1;
        if(pos<=mid)modify(pos, k, ql, mid, rt<<1);
        else modify(pos, k, mid+1, qr, rt<<1|1);
        pushup(rt);
    }
    ll query(int l, int r, int ql, int qr, int rt)
    {
        if(l<=ql&&qr<=r)
        {
            return a[rt];
        }
        int mid=(ql+qr)>>1;
        ll ans=-1;
        if(l<=mid)ans=max(ans, query(l, r, ql, mid, rt<<1));
        if(r>mid)ans=max(ans, query(l, r, mid+1, qr, rt<<1|1));
        return ans;
    }
}tree;
```

比h[i]小的最大值

酷哥创客

感谢您的观看

